

understanding

JAVA CLASS & OBJECT

WHAT IS A CLASS?

- ✗ A class is a blueprint or prototype from which objects are created.
- ✗ It defines the *state* (fields/variables) and *behavior* (methods) that every object made from it will have.
- ✗ A class does not itself hold data — it's a template.
- ✗ An object (an *instance* of the class) is what actually holds data at runtime.

```
class MyClass {  
    // class body  
}
```

ACCESS CONTROL

- ✗ Java has four access levels for controlling who can see/use a class member:

Modifier	Access Level
public	Visible everywhere
protected	Visible in the same package, and to subclasses elsewhere
(no modifier / package-private)	Visible only within the same package
private	Visible only within the declaring class itself

- ✗ **Good practice (encapsulation):** make fields private and provide public getter/setter methods to control access — this protects an object's internal state from unrestricted outside interference.

FIELDS (MEMBER VARIABLES)

- ✗ Fields store the state of an object.
- ✗ Each instance of a class gets its own copy of instance fields — that's why two Bicycle objects can have different speed values at the same time.

Example:

```
public class Bicycle {  
    // fields (state)  
    int cadence;  
    int gear;  
    int speed;  
}
```

CLASS MEMBERS VS. INSTANCE MEMBERS

- ✗ Instance variables/methods belong to each object separately — every object has its own copy.
- ✗ Class variables/methods (declared with the static keyword) belong to the class itself, not to any one object. There is only one copy, shared by all instances.

```
static int numberOfBicycles = 0; // a class (static) variable
```

```
static int maxSpeed = 50; // a class (static) variable
```

You can access a static member either through an object reference or, more conventionally, directly through the class name: **Bicycle.numberOfBicycles**.

METHODS

- Methods define behavior — what an object can *do*.
- A method declaration has six components:
 - Modifiers (e.g., public, private)
 - Return type (or void if nothing is returned)
 - Method name
 - Parameter list in parentheses
 - An optional list of exceptions to throw
 - The method body, enclosed in brace
- ```
public int salCalculate() {
 • //method body
 }
```
- ✗ **Returning a value:** Every method must declare its return type; if it returns nothing, the return type is void.
- ✗ Any method that isn't declared void must contain a return statement.

# WHAT IS AN OBJECT?

---

- ✗ If a class is the blueprint, an object is the actual thing built from that blueprint.
- ✗ A typical Java program creates many objects, and those objects interact by invoking methods on each other
- ✗ Every object has three basic characteristics:
  - State — the values held in its fields at any given moment
  - Behavior — what it can do, exposed through its methods
  - Identity — a unique existence in memory, distinct from other objects even if their state is identical



# **BITS AND BYTES**

---

- ✖ Bits and bytes are the smallest units of data in a computer.
- ✖ bit is a single binary digit, with a value of either 0 or 1.
- ✖ A byte is a group of 8 bits.
- ✖ 1 byte = 8 bits (01011011)
- ✖ In computer memory, a bit is stored as electrical voltage, where a voltage above a certain threshold represents a 1, and a voltage below that threshold represents a 0.
- ✖ In hard disk drives, a bit is stored as magnetism, where an area magnetized in one orientation represents a 1, and a magnetized area in the opposite orientation represents a 0.



# BITS AND BYTES

---

- What is a Byte?
- A byte is a group of 8 bits, like 10001011.
- Each bit can be either 0 or 1, and with 8 bits in a byte, there are  $2^8 = 256$  different values a byte can have.
- Using one byte, we can store:
  - A pixel with one out of 256 different colors.
  - An unsigned number from 0 to 255.
  - A signed number from -128 to 127.
  - A character from the ASCII table.

# DATA TYPE

---

- ✗ Data type specify the different type of values that are stored on the variables.
- ✗ Java is a **statically-typed language** — every variable must be declared with a type *before* it can be used.
- ✗ Declaring a variable means stating both its **type** and its **name**:

`int gear = 1;`

- ✗ A variable's data type determines two things:
  1. What **values** it may hold
  2. What **operations** may be performed on it

Types: Primitive and non-primitive(reference)

# DATA TYPES

---

✗ There are 8 built-in primitive data types

byte – 1 byte -- (-128) to 127 | ( $2^8 = 256$ ) (0 to 255 unsigned)

int – 4 byte – (-2147483648) to 2147483647

float – 4 byte –

long – 8 byte --

double – 8 byte --

char – 2 byte --

short – 2 byte – (-32768) to 32767

boolean – not specified – true, false

✗ Classes, arrays, and String are non-primitive/ reference types in Java.

# VARIABLES

---

- ✗ Variables are containers for storing data values.
- 1. **Local variable:** Declared inside a method, constructor, or block. Just as an object stores state in fields, a method stores its temporary working state in local variables:
- 2. **Instance variable:** Declared inside a class but outside methods, and without static. each *instance* (object) of the class gets its own independent copy.
- 3. **Static variable / Class variable:** Declared inside a class with the static keyword. A class variable is any field declared with the static modifier. This **one copy** of the variable, no matter how many objects of the class are created.
- 4. **Final :** cant change the value.

# RULES TO DEFINE VARIABLES

---

- ✗ A variable name can consist of Capital letters A-Z, lowercase letters a-z, digits 0-9, and two special characters such as \_ underscore and \$ dollar sign.
- ✗ The first character must not be a digit.
- ✗ Blank spaces cannot be used in variable names.
- ✗ Java keywords cannot be used as variable names.
- ✗ Variable names are case-sensitive.
- ✗ There is no limit on the length of a variable name but by convention, it should be between 4 to 15 chars.
- ✗ Variable names always should exist on the left-hand side of assignment operators.

# OPERATOR

---

- ✖ Operators are special symbols that perform an operation on one, two, or three operands, and then return a result.

1. Arithmetic operators- Used for mathematical calculations.

- + addition
- Subtraction
- \* Multiplication
- / Division
- % Modulus/remainder

# OPERATOR

## 2. Relational / Comparison operators

| Operator | Meaning                  |
|----------|--------------------------|
| ==       | Equal to                 |
| !=       | Not equal to             |
| >        | Greater than             |
| <        | Less than                |
| >=       | Greater than or equal to |
| <=       | Less than or equal to    |



# OPERATOR

---

## 3. Logical operators (Short circuit logical operator)

— Used to combine multiple conditions.

✗ **&& — AND** --Both conditions must be true.

✗ Example –

```
if (age >= 18 && age <= 60) {
 System.out.println("Valid age");
}
```

✗ **|| — OR** -- At least one condition must be true.

```
if (day == 6 || day == 7) {
 System.out.println("Weekend");
}
```

✗ **! — NOT** ---Reverses the boolean value.

```
boolean raining = false;
if (!raining) {
 System.out.println("Go outside");
}
```

# OPERATOR

4. Assignment operators – Used to assign values to variables.

```
int x = 10;
```

= means **assign**, not "equal to" in the mathematical sense.

Compound assignment operators

| Operator | Example | Equivalent to |
|----------|---------|---------------|
| =        | x = 5   | x = 5         |
| +=       | x += 5  | x = x + 5     |
| -=       | x -= 5  | x = x - 5     |
| *=       | x *= 5  | x = x * 5     |
| /=       | x /= 5  | x = x / 5     |
| %=       | x %= 5  | x = x % 5     |

# OPERATOR

---

## 5. Increment and decrement operators

**++** Increases a value by 1.

```
int x = 10;
x++; // new value = 11
```

**--** Decreases a value by 1.

```
int x = 10;
x--; // new value = 9
```

### ✖ Pre-increment vs post-increment

Post-increment --

```
int x = 10;
int y = x++;
```

//First x is assigned to y, then x increases.

New value--

```
y = 10
x = 11
```

# OPERATOR

---

## Pre-increment

```
int x = 10;
```

```
int y = ++x;
```

//First x increases, then it is assigned to y.

New value

x = 11

y = 11

# OPERATOR

---

## 6. Unary operators

Unary means the operator works on one operand.

`+x`

`-x`

`++x`

`--x`

`!x`

exp:

```
int x = 10;
```

```
System.out.println(-x);
```

o/p → -10

# OPERATOR

---

## 7. Ternary operator

The ternary operator is a short form of if-else.

**: ? :**

**condition ? valueIfTrue : valueIfFalse;**

**Example:**

```
int age = 20;
```

```
String result = (age >= 18) ? "Adult" : "Minor";
```

```
System.out.println(result);
```

o./p-- Adult

# OPERATOR

8. Bitwise operators– These operate at the bit level.

| Operator | Name        |
|----------|-------------|
| &        | Bitwise AND |
| ^        | Bitwise XOR |
| ~        | Bitwise NOT |

Example :

```
int a = 5;
```

```
int b = 3;
```

```
System.out.println(a & b);
```

O/p  $\rightarrow 5 \& 3 = 1$

Binary representation:

$5 \rightarrow 101$

$3 \rightarrow 011$

-----

$001 \rightarrow 1$



# OPERATOR

## 9. Shift operators

Used to shift bits left or right.

| Operator | Meaning              |
|----------|----------------------|
| <<       | Left shift           |
| >>       | Signed right shift   |
| >>>      | Unsigned right shift |

```
int x = 5;
```

```
System.out.println(x << 1);
```

5 → 101

↓ shift left

1010 → 10

o/p → 5 << 1 = 10

# OPERATOR

---

## 10. instanceof operator

Used to check whether an object belongs to a particular type.

```
String name = "CDAC";
System.out.println(name instanceof String);
```

o/p → true